Backpropagation: From Error to Credit

Chapter 3 · Local derivatives, multiple paths, linear layers, and automatic differentiation

Dr. A. Belcaid

Table of contents

1 Credit assignment **2**

2 Begin with one variable **3**
2.1 A derivative is local sensitivity . . . 3
2.2 One long expression, several small operations . . . 4
2.3 Backward messages . . . 5
2.4 Practice: draw first, differentiate second . . . 6

3 Several paths: add the contributions **7**
3.1 Regularization creates another path . . . 8

4 From one neuron to a linear layer **9**
4.1 Matrix form . . . 10

5 Backpropagation as message passing **10**

6 A backward-rule toolbox **11**

7 Complete two-layer binary classifier **12**

8 Automatic differentiation **12**

9 What gradients reveal **13**

10 Backpropagation in five moves **14**

11 Exercises **14**
11.1 Exercise 1: shared paths . . . 14
11.2 Exercise 2: linear-layer shapes . . . 14
11.3 Exercise 3: accumulation bug . . . 14

12 Further reading **15**

PART II · NEURAL NETWORKS FROM SCRATCH

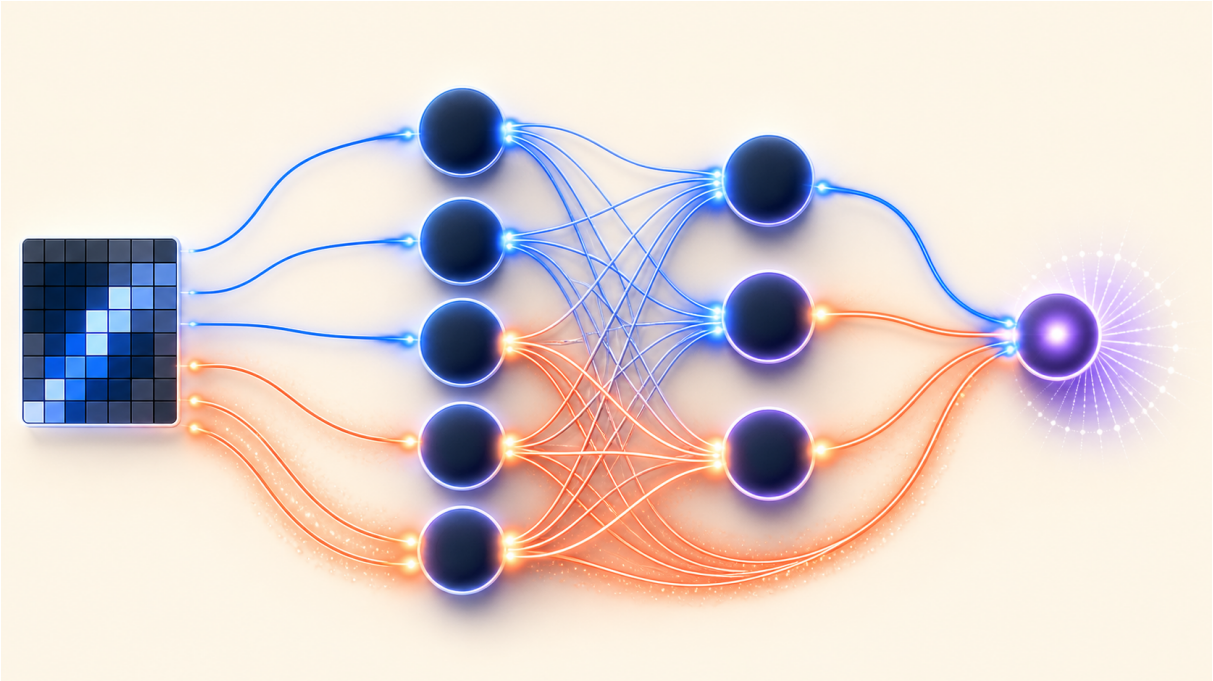


Figure 1: Forward signals and backward credit messages travel through a multilayer network. Original course illustration generated for this chapter.

A network can produce a prediction and a loss without knowing how any parameter contributed to the result. Backpropagation answers the missing question: **if the loss changed, which earlier values were responsible, in which direction, and by how much?**

By the end of this chapter, you should be able to

1. Explain backpropagation as credit assignment on a computation graph.
2. Apply the chain rule through a sequence of scalar operations.
3. Add gradient contributions when a value reaches the loss through several paths.
4. Derive backward rules for a linear layer, common activations, and classification losses.
5. Trace the complete backward pass of a two-layer binary classifier.
6. Explain how reverse-mode automatic differentiation executes local backward rules.
7. Interpret gradient signs, magnitudes, and layerwise flow without yet choosing an optimizer.

1 Credit assignment

Consider a binary classifier that confidently predicts “dog” for a cat image. The final loss is one number, but the computation that produced it involved every input feature, weight, bias, activation, and intermediate representation. Changing only the loss does not tell us where the error came from.

The **credit-assignment problem** is to distribute responsibility for the final loss across the values that produced it. For a parameter w, the derivative

$$\frac{\partial \mathcal{L}}{\partial w}$$

measures the local sensitivity of the loss to that parameter. Its sign indicates direction; its magnitude indicates strength.

Backpropagation is an efficient procedure for computing these sensitivities. It is the reverse-mode chain rule organized around a computation graph (Rumelhart et al. 1986).

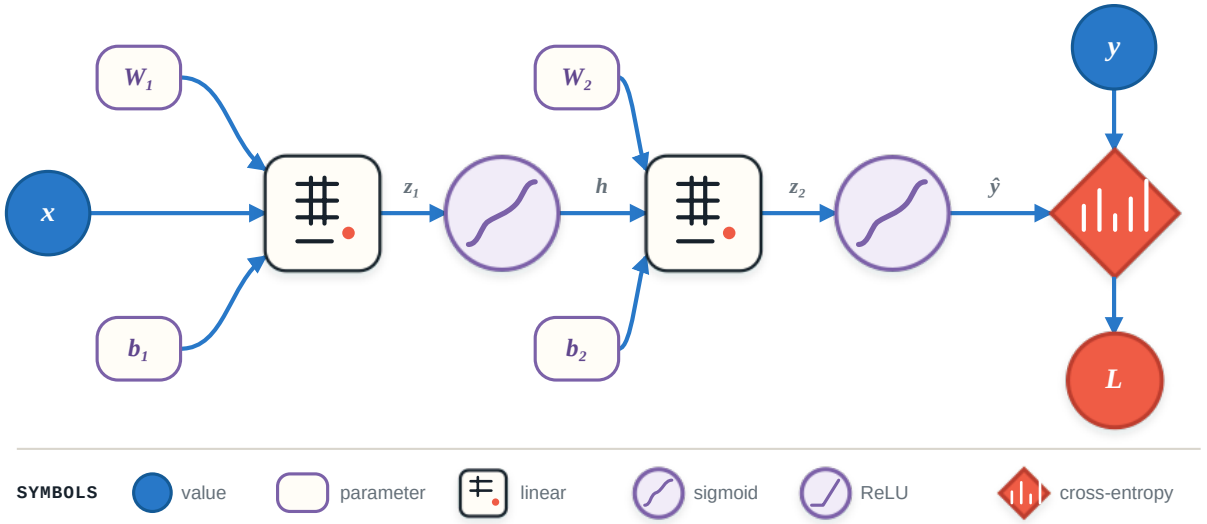


Figure 2: The symbolic graph used throughout the lecture. Values are blue circles, parameters are purple rounded rectangles, affine maps are square blocks, activations are purple circles, and the loss is coral.

The graph separates two ideas:

- **identity:** short labels such as x, W_1, h, and $haty$ name values;
- **operation:** stable symbols identify affine maps, activations, and losses without filling every node with prose.

2 Begin with one variable

2.1 A derivative is local sensitivity

For a scalar function $y = f(x)$, the derivative at x is the limiting ratio

$$\frac{dy}{dx} = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}.$$

A derivative is a local promise: for a sufficiently small change,

$$\Delta y \approx \frac{dy}{dx}\Delta x.$$

The sign tells us whether y locally rises or falls as x increases. The magnitude tells us how sensitive y is to that change.

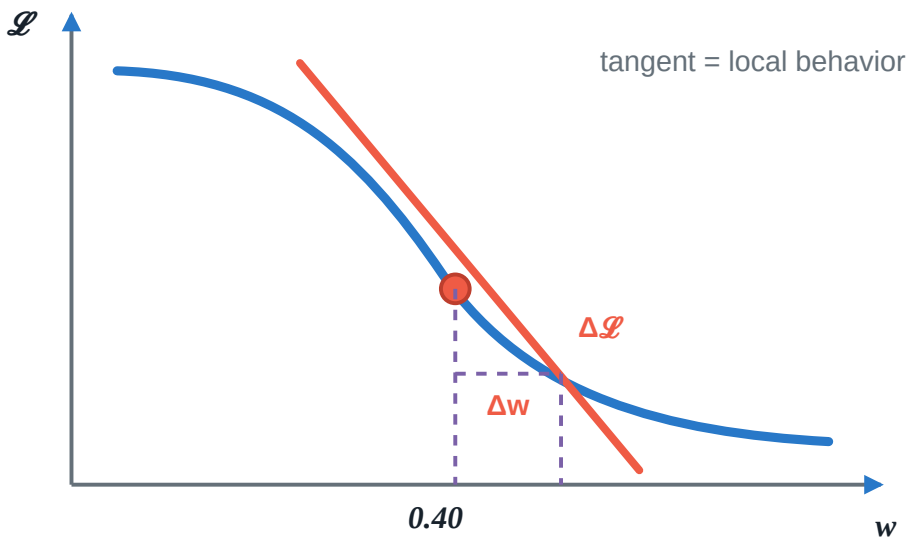


Figure 3: A local perturbation is compared with the tangent prediction at one point.

2.2 One long expression, several small operations

Take the expression

$$\mathcal{L}(x) = \left[(x^2+1)^3 - 2\right]^2, \qquad x = 1.$$

Direct differentiation gives

$$\frac{d\mathcal{L}}{dx} = 2\left[(x^2+1)^3 - 2\right] 3(x^2+1)^2\, 2x,$$

and therefore

$$\left.\frac{d\mathcal{L}}{dx}\right|_{x=1} = 288.$$

This is correct, but the nested algebra hides the reusable structure. Introduce intermediate values instead:

$$a = x^2, \qquad b = a + 1, \qquad c = b^3, \qquad d = c - 2, \qquad \mathcal{L} = d^2.$$

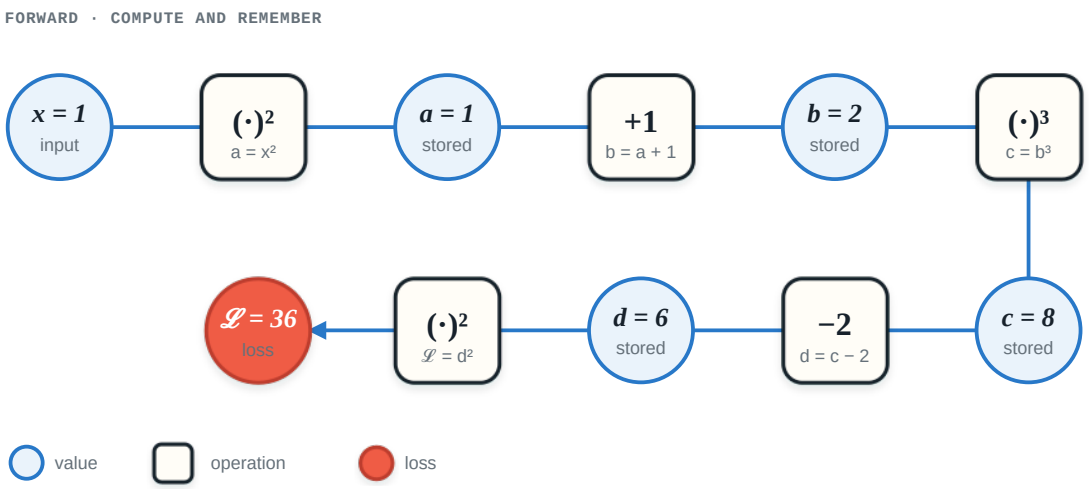


Figure 4: The forward graph evaluates the expression as five small operations and stores the intermediate values.

Each operation now knows one local derivative:

$$\frac{da}{dx} = 2x, \qquad \frac{db}{da} = 1, \qquad \frac{dc}{db} = 3b^2, \qquad \frac{dd}{dc} = 1, \qquad \frac{d\mathcal{L}}{dd} = 2d.$$

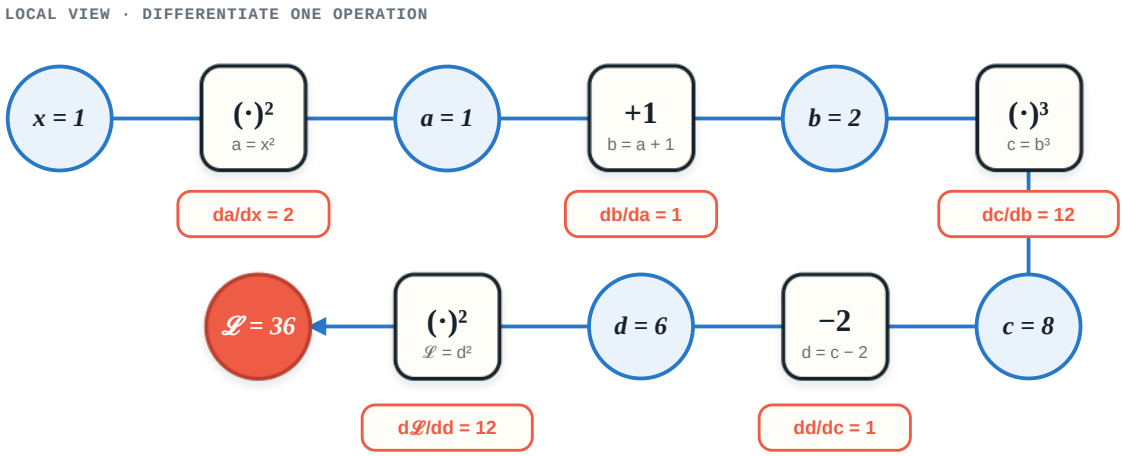


Figure 5: Every operation differentiates only its own input-output relationship.

2.3 Backward messages

Seed the final scalar with

$$\frac{d\mathcal{L}}{d\mathcal{L}} = 1.$$

Then move right to left. At every operation,

$$\boxed{\text{outgoing message} = \text{incoming message} \times \text{local derivative}.}$$

For the example,

$$\begin{aligned}
\frac{d\mathcal{L}}{dd} &= 1 \cdot 2d = 12, \\
\frac{d\mathcal{L}}{dc} &= 12 \cdot 1 = 12, \\
\frac{d\mathcal{L}}{db} &= 12 \cdot 3b^2 = 144, \\
\frac{d\mathcal{L}}{da} &= 144 \cdot 1 = 144, \\
\frac{d\mathcal{L}}{dx} &= 144 \cdot 2x = 288.
\end{aligned}$$

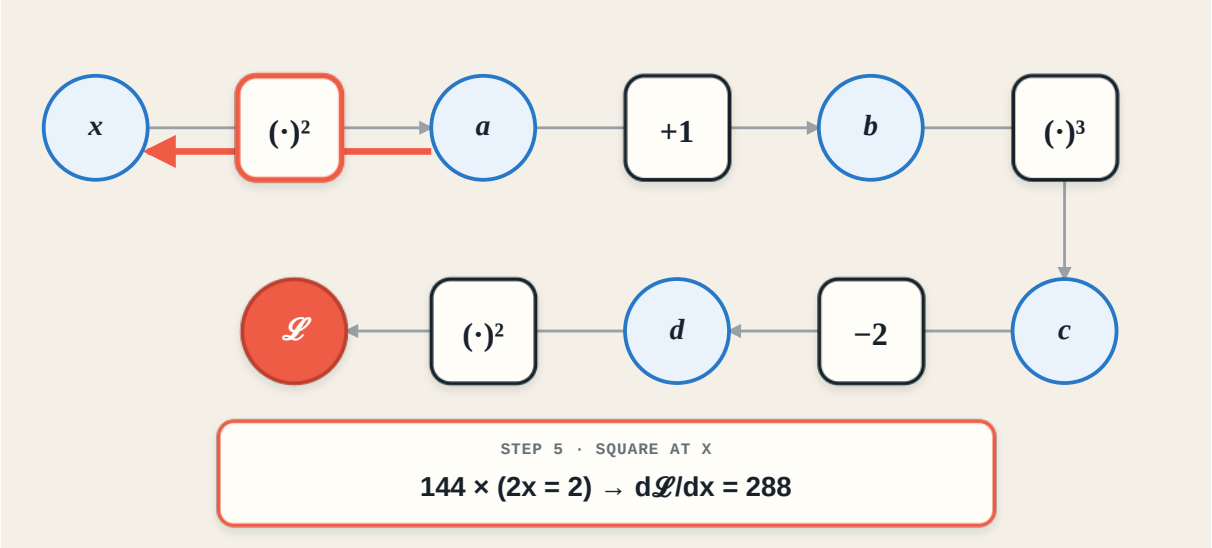


Figure 6: The final backward state highlights the local computation that sends the derivative 288 to x.

The result matches direct algebra, but the computation was assembled from local rules. This is the mechanism that scales.

2.4 Practice: draw first, differentiate second

For one sigmoid neuron,

$$z = wx + b, \qquad \hat{y} = \sigma(z), \qquad \mathcal{L} = \frac{1}{2}(\hat{y} - y)^2,$$

with $x = 2$, $w = 1$, $b = -1$, and $y = 1$:

1. Draw the computation graph.
2. Compute the stored forward values.
3. Backpropagate to w, b, and x.

Worked solution

The graph and forward values are shown in Figure 7. Backpropagation gives $g_{\hat{y}} \approx -0.269$, $g_z \approx -0.0529$, $\partial \mathcal{L}/\partial w \approx -0.1058$, and equal gradients of approximately -0.0529 for b and x.

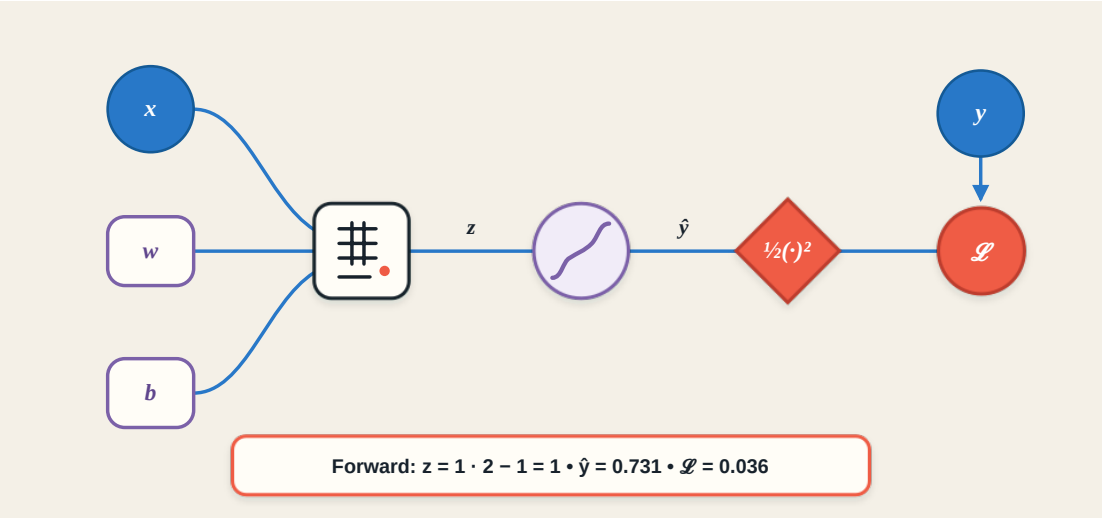


Figure 7: The completed sigmoid-neuron graph.

3 Several paths: add the contributions

Suppose f depends on x and y, while both x and y depend on t. Then t reaches f through two paths:

$$\boxed{\frac{df}{dt} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}.}$$

For

$$f(x,y) = y + e^{xy}, \qquad x(t) = \cos t, \qquad y(t) = t^2,$$

the local derivatives are

$$\frac{\partial f}{\partial x} = ye^{xy}, \qquad \frac{\partial f}{\partial y} = 1 + xe^{xy}, \qquad \frac{dx}{dt} = -\sin t, \qquad \frac{dy}{dt} = 2t.$$

Therefore

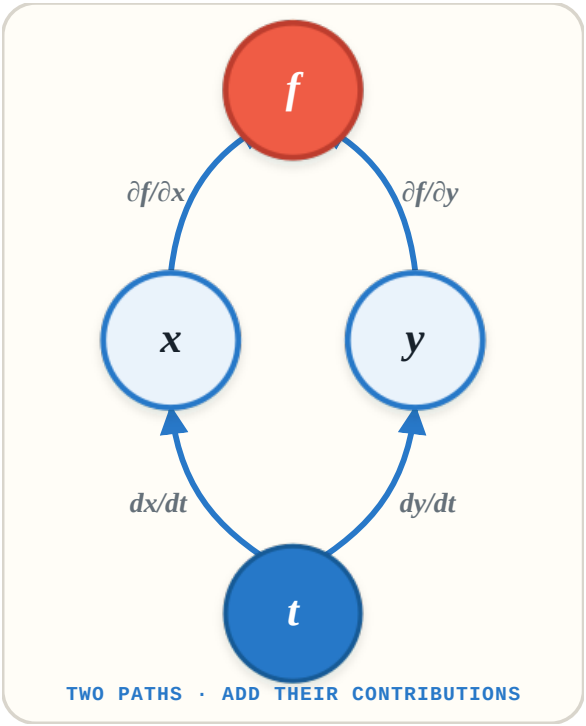


Figure 8: Two paths connect t to f. Each path contributes one product of local derivatives.

$$\frac{df}{dt} = -t^2 e^{t^2 \cos t} \sin t + 2t \left(1 + \cos t \, e^{t^2 \cos t}\right).$$

The plus sign is structural: when several paths connect a value to the loss, backpropagation adds their messages.

3.1 Regularization creates another path

For

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \frac{\lambda}{2} w^2,$$

the weight reaches the total loss through the prediction and through the penalty.

Hence

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial w} = \frac{\partial \mathcal{L}_{\text{data}}}{\partial w} + \lambda w.$$

Using the preceding neuron example with $\lambda = 0.1$ and $w = 1$ gives

$$-0.1058 + 0.1 = -0.0058.$$

The regularizer did not replace the data gradient. It contributed another message.

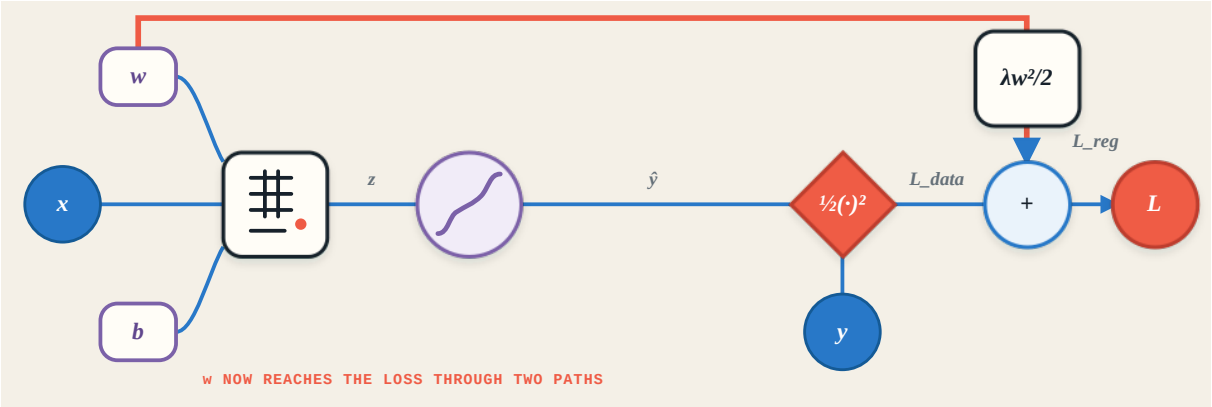


Figure 9: The weight w reaches the total loss through the data-loss path and the regularization path.

4 From one neuron to a linear layer

Consider a four-input, three-output affine layer followed by a scalar loss. Focus first on

$$h_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + w_{14}x_4 + b_1.$$

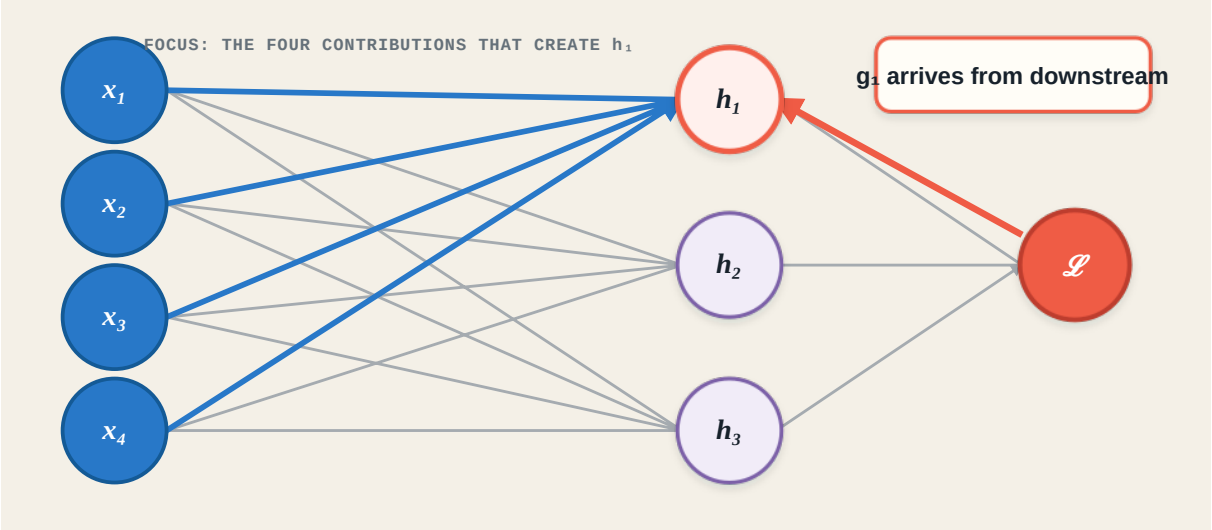


Figure 10: A four-to-three linear layer. The four edges that produce h1 and the incoming message from the loss are highlighted.

Let

$$g_1 = \frac{\partial \mathcal{L}}{\partial h_1}.$$

The messages sent by h_1 are

$$\frac{\partial \mathcal{L}}{\partial w_{1j}} = g_1 x_j, \qquad \left.\frac{\partial \mathcal{L}}{\partial x_j}\right|_{h_1} = g_1 w_{1j}, \qquad \frac{\partial \mathcal{L}}{\partial b_1} = g_1.$$

The input x_j is used by all three output neurons, so it adds three contributions:

$$\frac{\partial \mathcal{L}}{\partial x_j} = \sum_{i=1}^{3} \frac{\partial \mathcal{L}}{\partial h_i} w_{ij}.$$

4.1 Matrix form

For

$$\mathbf{h} = W\mathbf{x} + \mathbf{b},$$

with incoming gradient $\mathbf{g}_h = \partial \mathcal{L}/\partial \mathbf{h}$, the linear-layer rules are

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = W^\top \mathbf{g}_h}, \qquad \boxed{\frac{\partial \mathcal{L}}{\partial W} = \mathbf{g}_h \mathbf{x}^\top}, \qquad \boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \mathbf{g}_h}.$$

The dimensions explain the forms:

- transpose to send one accumulated message to each input;
- outer product to create one gradient for every weight;
- copy the incoming vector because each bias has local derivative one.

5 Backpropagation as message passing

In the forward graph, the **parents** of a node produced its value and its **children** consumed it. Backpropagation reverses this direction.

At a node v:

1. receive messages from every child;
2. add them to form g_v;
3. for each parent p_i, compute

$$\text{message}(p_i) = g_v \frac{\partial v}{\partial p_i};$$

4. add that message to the parent's gradient accumulator.

This language unifies scalar calculus, matrix layers, and automatic differentiation.

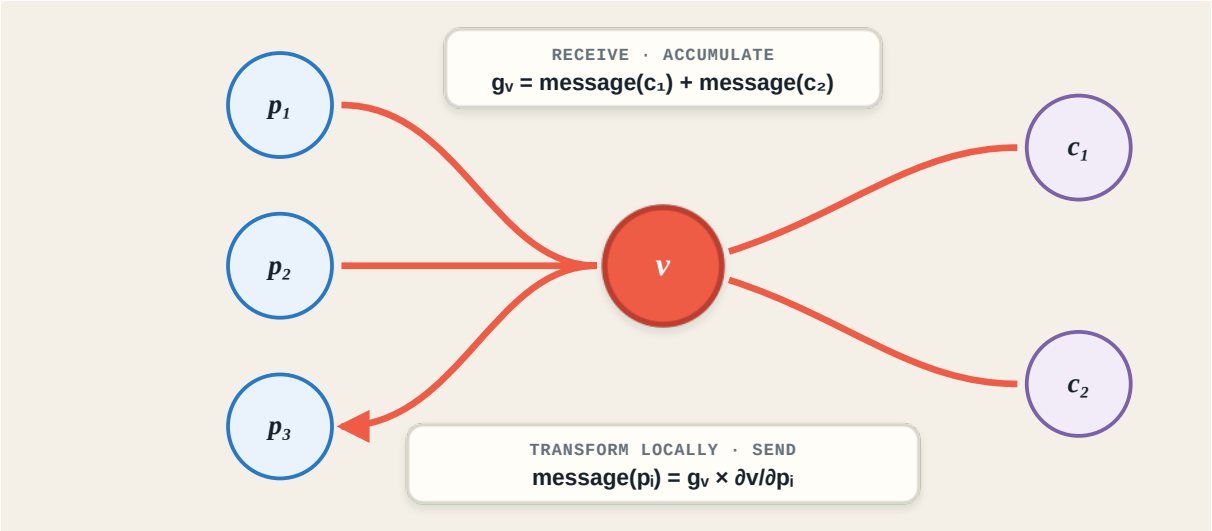


Figure 11: A node receives messages from its children, adds them, and sends locally transformed messages to its parents.

6 A backward-rule toolbox

Let g_y denote the gradient arriving at a block's output. The following rules are worth recognizing:

Block	Forward	Backward message
Linear	$\mathbf{y} = W\mathbf{x} + \mathbf{b}$	$\mathbf{g}_x = W^\top \mathbf{g}_y$, $\nabla_W \mathcal{L} = \mathbf{g}_y \mathbf{x}^\top$, $\nabla_b \mathcal{L} = \mathbf{g}_y$
Sigmoid	$y = \sigma(x)$	$g_x = g_y\, y(1-y)$
Tanh	$y = \tanh(x)$	$g_x = g_y(1-y^2)$
ReLU	$y = \max(0, x)$	$g_x = g_y \mathbf{1}[x > 0]$
Softmax	$p_i = e^{z_i} / \sum_j e^{z_j}$	$\mathbf{g}_z = \mathbf{p} \odot (\mathbf{g}_p - \mathbf{p}^\top \mathbf{g}_p)$
Cross-entropy	$\mathcal{L} = -\sum_i y_i \log p_i$	$\mathbf{g}_p = -\mathbf{y}/\mathbf{p}$

Two fused shortcuts occur constantly in classifiers:

$$\boxed{\text{softmax + cross-entropy: } \mathbf{g}_z = \mathbf{p} - \mathbf{y}}$$

and

$$\boxed{\text{sigmoid + binary cross-entropy: } g_z = \hat{y} - y.}$$

Softmax differs from sigmoid, tanh, and ReLU because its outputs interact: changing one logit changes every probability.

7 Complete two-layer binary classifier

For a hidden sigmoid layer and sigmoid binary output,

$$\begin{aligned}
\mathbf{z}_1 &= W_1\mathbf{x} + \mathbf{b}_1, \\
\mathbf{h} &= \sigma(\mathbf{z}_1), \\
z_2 &= W_2\mathbf{h} + b_2, \\
\hat{y} &= \sigma(z_2), \\
\mathcal{L} &= -[y \log \hat{y} + (1-y)\log(1-\hat{y})].
\end{aligned}$$

The backward pass is

$$\begin{aligned}
g_{z_2} &= \hat{y} - y, \\
\nabla_{W_2}\mathcal{L} &= g_{z_2}\mathbf{h}^\top, & \nabla_{b_2}\mathcal{L} &= g_{z_2}, \\
\mathbf{g}_h &= W_2^\top g_{z_2}, \\
\mathbf{g}_{z_1} &= \mathbf{g}_h \odot \mathbf{h} \odot (1-\mathbf{h}), \\
\nabla_{W_1}\mathcal{L} &= \mathbf{g}_{z_1}\mathbf{x}^\top, & \nabla_{\mathbf{b}_1}\mathcal{L} &= \mathbf{g}_{z_1}, \\
\mathbf{g}_x &= W_1^\top \mathbf{g}_{z_1}.
\end{aligned}$$

The complete companion derivation expands every step, checks all tensor shapes, explains the stable sigmoid–BCE simplification, and covers mini-batches. A PDF version is available for offline study.

8 Automatic differentiation

Automatic differentiation does not replace the chain rule. It records enough information during the forward pass to execute these local rules automatically in reverse order.

Karpathy's micrograd is a deliberately small scalar reverse-mode autodiff engine. It builds a dynamic directed acyclic graph and provides a PyTorch-like `Value` interface suitable for study.

```
from micrograd.engine import Value

x = Value(1.0)
a = x**2
b = a + 1
c = b**3
d = c - 2
L = d**2

L.backward()
assert x.grad == 288.0
```

Each `Value` stores conceptually:

Property	Purpose
`data`	forward value
`grad`	accumulated gradient messages
`_prev`	parent nodes
`_op`	operation that created the value
`_backward`	operation-specific local backward rule

For example,

```
x, y, z = Value(2.0), Value(3.0), Value(1.0)
a = x * y       # data=6,  _op='*',   _prev={x, y}
b = a + z       # data=7,  _op='+',   _prev={a, z}
L = b ** 2      # data=49, _op='**2', _prev={b}
```

Calling `backward()` seeds `L.grad = 1`, computes a topological ordering, and visits the nodes in reverse. The central update is

```
parent.grad += node.grad * local_derivative
```

The `+=` is essential. Assignment would erase previous contributions whenever a value is reused through several paths. The coding session will implement this `Value` abstraction from scratch; no framework machinery is required.

9 What gradients reveal

A gradient should be interpreted before it is used:

- **Sign:** which local direction raises the loss?
- **Magnitude:** how sensitive is the loss to a small change?
- **Near zero:** possible local irrelevance, saturation, or cancellation between paths.

A near-zero gradient is a clue, not a complete diagnosis. Forward activations and the graph structure provide the missing context.

Across layers, gradient magnitudes can reveal qualitative patterns:

- **healthy flow:** earlier layers continue to receive measurable messages;
- **vanishing gradients:** repeated small local derivatives progressively shrink messages;
- **exploding gradients:** repeated large transformations progressively amplify them.

Gradients expose symptoms. Initialization, architecture, normalization, and optimization determine what to do about them in later chapters.

10 Backpropagation in five moves

1. **Compute forward:** store the values needed by local rules.
2. **Seed the loss:** set $g_{\mathcal{L}} = 1$.
3. **Move backward:** visit nodes in reverse topological order.
4. **Transform locally:** incoming gradient times local derivative.
5. **Add paths:** accumulate every contribution; never overwrite.

> **The whole idea**
>
> **Receive messages. Add them. Apply the local rule. Send messages to parents.**

11 Exercises

11.1 Exercise 1: shared paths

Let

$$u = xy, \qquad v = x + y, \qquad \mathcal{L} = u^2 + v^2.$$

Draw the graph and compute $\partial \mathcal{L}/\partial x$ and $\partial \mathcal{L}/\partial y$ at $x = 2, y = 3$. Identify where messages must be added.

> **Solution**
>
> $\partial \mathcal{L}/\partial x = 2u\,y + 2v = 46$ and $\partial \mathcal{L}/\partial y = 2u\,x + 2v = 34$. Each variable reaches the loss through both u and v, so the two path contributions add.

11.2 Exercise 2: linear-layer shapes

Suppose $W \in \mathbb{R}^{5\times 3}$, $\mathbf{x} \in \mathbb{R}^3$, and $\mathbf{h} = W\mathbf{x} + \mathbf{b}$. If $\mathbf{g}_h \in \mathbb{R}^5$, state the expression and shape of the gradients with respect to W, $\mathbf{b}$, and $\mathbf{x}$.

> **Solution**
>
> $\nabla_W \mathcal{L} = \mathbf{g}_h \mathbf{x}^\top \in \mathbb{R}^{5\times 3}$, $\nabla_{\mathbf{b}} \mathcal{L} = \mathbf{g}_h \in \mathbb{R}^5$, and $\mathbf{g}_x = W^\top \mathbf{g}_h \in \mathbb{R}^3$.

11.3 Exercise 3: accumulation bug

A student implements

```
parent.grad = node.grad * local_derivative
```

instead of using `+=`. Give the smallest computation graph for which this produces a wrong result, and explain which message is lost.

> **Solution**
>
> For $\mathcal{L} = x + x$, both addition inputs refer to the same `Value`. Assignment leaves a gradient of 1; accumulation correctly produces $1 + 1 = 2$.

12 Further reading

- Rumelhart, Hinton, and Williams introduced the influential modern presentation of learning representations through back-propagated errors (Rumelhart et al. 1986).
- Roger Grosse's CSC321 Lecture 6 develops the same scalar-to-network progression used in this chapter.
- Stanford CS231n's Backpropagation, Intuitions emphasizes computation graphs, local gradients, and reusable gate patterns.
- Karpathy's micrograd is a compact reference implementation of scalar reverse-mode autodiff.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. "Learning Representations by Back-Propagating Errors." *Nature* 323: 533–36. https://doi.org/10.1038/323533a0.